EX.NO: 2

Implementation of Runtime Polymorphism Using Virtual Functions and Abstract Classes

Program

```
#include <iostream>
using namespace std;

class Shape
{
public:
    virtual float area() = 0; // Pure virtual function
};

class Circle : public Shape
{
private:
    float radius;
public:
    Circle(float r)
    {
        radius = r;
    }
    float area()
    {
        return 3.14 * radius * radius;
    }
};

class Rectangle : public Shape
{
private:
    float length, breadth;
public:
    Rectangle(float l, float b)
    {
        length = l;
        breadth = b;
    }
    float area()
    {
        return length * breadth;
    }
};

class Triangle : public Shape
```

```
{
private:
    float base, height;
public:
    Triangle(float b, float h)
    {
        base = b;
        height = h;
    }
    float area()
    {
        return 0.5 * base * height;
    }
};

int main()
{
    Circle c(5);
    Rectangle r(4, 6);
    Triangle t(10, 8);

    Shape *ptr;

    ptr = &c;
    cout << "Area of Circle: " << ptr->area() << endl;

    ptr = &r;
    cout << "Area of Rectangle: " << ptr->area() << endl;

    ptr = &t;
    cout << "Area of Triangle: " << ptr->area() << endl;

    return 0;
}
```

Output

```
Area of Circle: 78.5
Area of Rectangle: 24
Area of Triangle: 40
```